INSTITUTO FEDERAL DO PARANÁ

MATHEUS CARSTENS DE OLIVEIRA

AUTOMAÇÃO DE ALERTA DE LOGS CRÍTICOS EM TEMPO REAL UTILIZANDO PLATAFORMA LOW-CODE N8N E INTERAÇÃO COM TELEGRAM

CURITIBA

2025

MATHEUS CARSTENS DE OLIVEIRA

AUTOMAÇÃO DE ALERTA DE LOGS CRÍTICOS EM TEMPO REAL UTILIZANDO PLATAFORMA LOW-CODE N8N E INTERAÇÃO COM TELEGRAM

Trabalho de Conclusão de Matéria apresentado ao professor Sandro de Oliveira como requisito para a conclusão da matéria de Interação Humano-Computador.

CURITIBA
2025

Resumo

O cenário atual de desenvolvimento *ágil* e de operações (DevOps) exige a rápida identificação e resposta a falhas para reduzir o Tempo Médio para Recuperação (**MTTR**). O problema central desta pesquisa reside na vulnerabilidade inerente aos sistemas de notificação de erros que dependem de processos manuais de monitoramento, resultando em latência crítica. Este trabalho teve como objetivo desenvolver e validar um *workflow* de automação na plataforma *low-code* **n8n** para monitoramento e notificação em tempo real de logs de erros. A metodologia empregou uma arquitetura **Trigger-Action** simples, onde o nó **Webhook** atua como gatilho de escuta para um *payload* JSON, que é processado e formatado pelo nó **Edit Fields** e, finalmente, enviado como alerta instantâneo via **Telegram**. Os resultados comprovaram a eficácia do *workflow*, que opera com latência de menos de um segundo, fornecendo informações acionáveis diretamente à equipe de resposta a incidentes, conforme demonstrado no *output* de notificação. Conclui-se que o uso da abordagem *low-code* se provou uma solução ágil e eficiente, capaz de eliminar a latência crítica do processo manual e reduzir significativamente o MTTR em ambientes DevOps.

Palavras-chave: *Low-Code*; DevOps; MTTR; Automação; n8n; Logs.

ABSTRACT

The current scenario of agile development and operations (DevOps) demands rapid identification of and response to failures to reduce the Mean Time to Recover (**MTTR**). The central problem of this research lies in the inherent vulnerability of error notification systems that rely on manual monitoring processes, resulting in critical latency. This work aimed to develop and validate an automation workflow on the **n8n low-code platform** for real-time critical log error monitoring and notification. The methodology employed a simple **Trigger-Action** architecture, where the **Webhook** node acts as a listening trigger for a JSON *payload*, which is processed and formatted by the **Edit Fields** node, and finally sent as an instant alert via **Telegram**. The results proved the effectiveness of the workflow, which operates with less than one second of latency, providing actionable information directly to the incident response team, as demonstrated in the notification output. It is concluded that the use of the *low-code* approach proved to be an agile and efficient solution, capable of eliminating critical latency from the manual process and significantly reducing MTTR in DevOps environments.

Keywords: Low-Code; DevOps; MTTR; Automation; n8n; Logs.

LISTA DE ILUSTRAÇÕES

Figura 1 – Arquitetura do Workflow completo no n8n. 10

Figura 2 – Configuração do Nó Webhook e Recebimento do Payload JSON. 11

Figura 3 – Configuração e Expressão do Nó Edit Fields para Formatação. 12

Figura 4 – Alerta Crítico Recebido no Canal do Telegram (Resultado Final). 14

Resumo 3
ABSTRACT 3
LISTA DE ILUSTRAÇÕES 4
1. Introdução 6
Justificativa e Fundamentação Metodológica 6
Objetivos do Estudo 7
2. Fundamentação Teórica 8
2.1 O Paradigma DevOps e a Métrica MTTR 8
2.2 Monitoramento de Logs e sua Importância Crítica 8
2.3 Desenvolvimento Low-Code como Abordagem Metodológica 9
3. Metodologia 10
3.1 Arquitetura do Workflow 10
3.2 Configuração Nodal e Fluxo de Dados 11
3.2.1 Configuração do Nó de Gatilho (Webhook) 11
3.2.2 Processamento e Formatação de Dados (Edit Fields) 11
3.2.3 Nó de Ação (Telegram) 12
4. RESULTADOS E DISCUSSÃO 13
4.1 Validação do Fluxo (Workflow) e Alerta em Tempo Real 13
4.2 Redução da Latência e Eficiência Operacional 15
4.3 Desafios Técnicos e Escolha da Plataforma 15
REFERÊNCIAS 16

1. Introdução

No cenário atual de desenvolvimento de *software* ágil e em ambientes de Integração Contínua (CI) e Entrega Contínua (CD), a disponibilidade e a confiabilidade dos sistemas são métricas essenciais. A rápida identificação e resposta a falhas é crucial para mitigar o impacto de interrupções, sendo o **monitoramento de logs** uma das atividades fundamentais para a saúde da aplicação. Os logs são documentos com **descrições detalhadas das ações processadas, incluindo mensagens de falhas e erros**, permitindo que o administrador identifique a origem de problemas e agilize a solução **(RIBAS et al., 2023)**.

A automação das práticas de desenvolvimento e operações, característica da metodologia **DevOps**, é crucial para aumentar a qualidade do *software* e, principalmente, reduzir o **Tempo Médio para Recuperação (MTTR)**. Organizações que adotam práticas robustas conseguem implementar mudanças e **reduzir significativamente o tempo de recuperação após falhas (LEAL et al., 2024)**. No entanto, a análise contínua desses registros muitas vezes depende de intervenção humana para análise, resultando em latência crítica na emissão de alertas. O problema central desta pesquisa reside, portanto, na **vulnerabilidade inerente aos sistemas de notificação de erros que dependem de processos manuais** para alimentar o ciclo de feedback essencial ao DevOps, comprometendo o MTTR.

Justificativa e Fundamentação Metodológica

Para endereçar a latência do processo manual, esta pesquisa utiliza a abordagem **Low-Code**. Esta metodologia representa uma solução inovadora que visa **acelerar a produção por meio de ambientes de trabalho intuitivos e componentes pré-construídos**, possibilitando a criação rápida de processos automatizados e a diminuição de prazos **(MOURA, 2025)**.

Essa escolha metodológica é estratégica, pois o uso de *builders* visuais e a **integração nativa com APIs externas** simplifica a conexão do *workflow* com serviços de mensageria (Telegram) e a recepção de dados de *logs*. A plataforma **n8n** surge como uma ferramenta *low-code* ideal, capaz de orquestrar a captura de dados de erro via **Webhook** e sua subsequente transformação em alertas acionáveis, garantindo a agilidade demandada pelo ambiente de desenvolvimento.

Objetivos do Estudo

Este estudo propõe o desenvolvimento de um *workflow* de automação capaz de monitorar, processar e notificar logs de erros críticos em tempo real.

O **Objetivo Geral** é criar uma ponte automatizada entre a ocorrência de uma falha em uma aplicação e a notificação imediata da equipe de desenvolvimento.

Os **Objetivos Específicos** incluem:

1. Configurar um gatilho **Webhook** para recepção do *payload* JSON de logs de erro de uma aplicação.
2. Utilizar o nó **Edit Fields** (Set) para manipular e formatar o log bruto em uma mensagem de alerta concisa (**alertMessage**).
3. Integrar a automação com o serviço **Telegram** para garantir notificação instantânea.
4. Analisar o ganho de eficiência do processo automatizado em relação ao método manual.

2. Fundamentação Teórica

A solução de automação proposta neste trabalho está alinhada a conceitos e práticas fundamentais da Engenharia de *Software* e da Gestão de Operações de Tecnologia da Informação. O presente capítulo visa estabelecer a base teórica para o desenvolvimento do *workflow*, definindo os conceitos de agilidade operacional, a importância dos registros de sistema e a metodologia de construção adotada.

2.1 O Paradigma DevOps e a Métrica MTTR

A metodologia DevOps busca aprimorar a colaboração e a integração entre as equipes de Desenvolvimento (*Dev*) e Operações (*Ops*), visando a entrega contínua de *software* de alta qualidade (LEAL et al., 2024). Um dos pilares centrais desta abordagem é a **automação** de processos, que permite que as organizações reduzam significativamente o retrabalho e o tempo gasto em tarefas manuais, aumentando a produtividade e a qualidade. A adoção dessas práticas robustas tem um impacto direto na capacidade da organização de responder a falhas.

A agilidade de resposta é medida por métricas como o **Mean Time To Recovery (MTTR)**, que quantifica o tempo necessário para que um sistema seja restaurado à sua operação normal após uma indisponibilidade. A rapidez na recuperação é crucial, pois, segundo a literatura, organizações que adotam práticas robustas de DevOps conseguem não apenas implementar mudanças mais rapidamente, mas também **reduzir significativamente o tempo de recuperação após falhas** (LEAL et al., 2024). Portanto, qualquer latência no sistema de notificação compromete diretamente a eficiência operacional e o MTTR.

2.2 Monitoramento de Logs e sua Importância Crítica

A base para qualquer sistema de resposta a incidentes é o registro de eventos. Os **logs** são definidos como documentos contendo **descrições detalhadas das ações processadas, incluindo falhas e erros**, com dados precisos de data e hora (RIBAS et al., 2023). A importância desses registros é crítica na

área de TI, pois eles permitem que o administrador ou a equipe de desenvolvimento identifique a origem exata de problemas e agilize a implementação de soluções.

Além de auxiliar na solução pontual de *bugs*, o monitoramento de logs é essencial para a saúde geral do *software*, pois permite a visualização do desempenho do sistema e a detecção precoce de atividades maliciosas. Em ambientes conteinerizados ou de alta escala, onde o isolamento de falhas é uma característica fundamental, a coleta e a análise imediata desses logs são cruciais para a estabilidade de todo o ambiente (RIBAS et al., 2023).

2.3 Desenvolvimento Low-Code como Abordagem Metodológica

Para a construção da solução proposta, foi adotada a abordagem **Low-Code**. O *low-code* representa uma metodologia inovadora que visa **acelerar a produção de *software* por meio de ambientes de trabalho intuitivos e componentes pré-construídos**, eliminando a necessidade de escrita manual de código complexo (MOURA, 2025).

A escolha de plataformas *low-code* como o n8n justifica-se pela sua capacidade de viabilizar a **criação rápida de processos automatizados e a diminuição de prazos**, o que é um fator decisivo em um contexto DevOps que exige agilidade na entrega. Outra vantagem estratégica é a **integração nativa com APIs externas** e a automação de lógica de negócios com *builders* visuais, simplificando a conexão do *workflow* com serviços de mensageria (Telegram) e sistemas de recebimento de logs (Webhook) (MOURA, 2025).

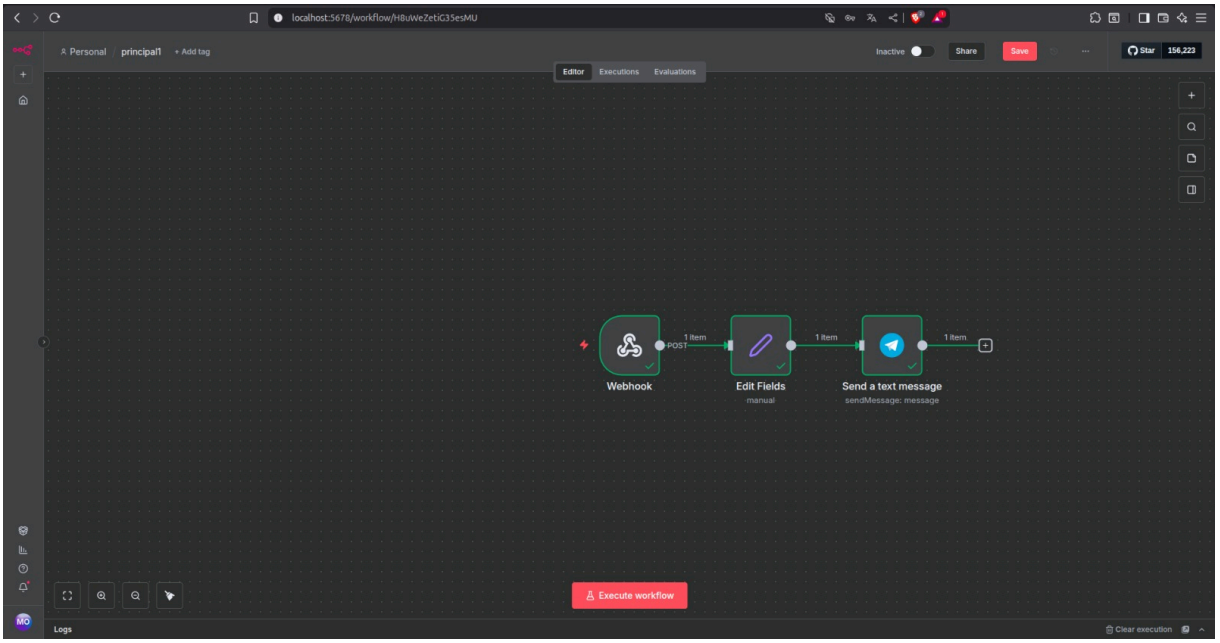
3. Metodologia

A metodologia adotada para o desenvolvimento do *workflow* de automação seguiu uma abordagem **exploratória** e de engenharia de processos, com foco na prototipação *low-code*. O ambiente de desenvolvimento foi a plataforma **n8n**, executada em ambiente local (Docker), utilizando-se o **Insomnia** para simulação do tráfego de logs. O sistema de notificação em tempo real escolhido foi o **Telegram**, dada sua facilidade de integração e baixo custo operacional.

3.1 Arquitetura do Workflow

O *workflow* foi estruturado na arquitetura básica de automação (Trigger-Action), visando a execução imediata após a ocorrência de um erro. A Figura 1 ilustra a arquitetura final do fluxo de dados, que consiste nas etapas de Recebimento, Processamento e Notificação.

Figura 1 – Imagem do *Workflow* completo.



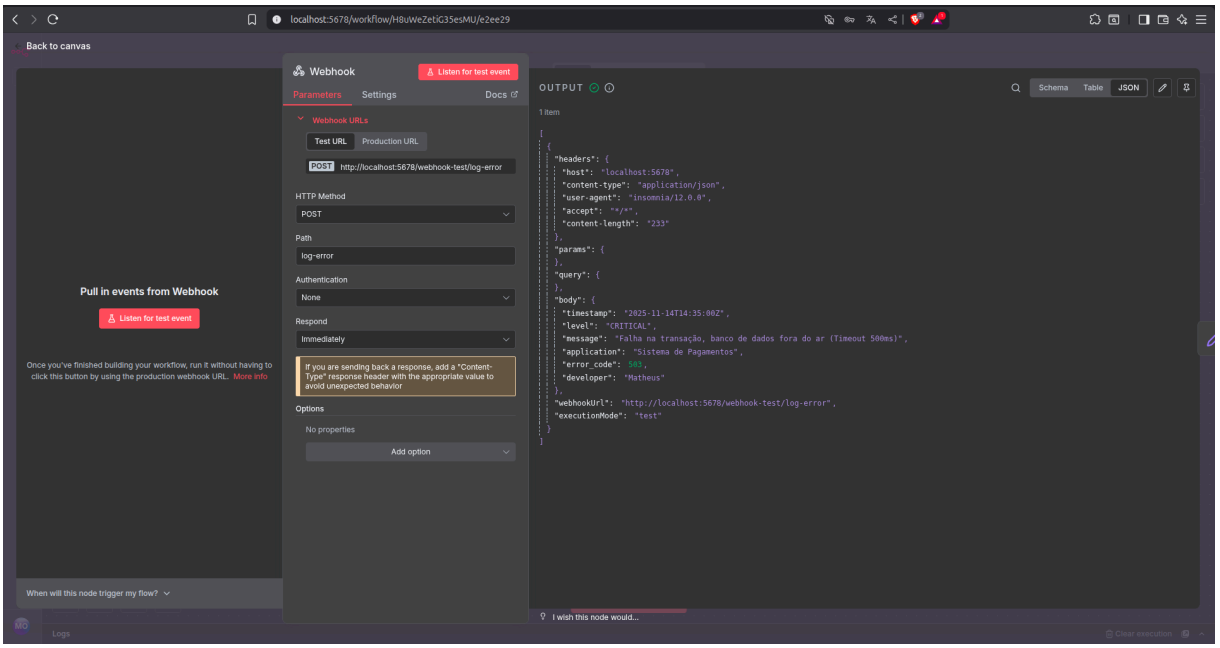
Fonte: Matheus Carstens de Oliveira (2025).

3.2 Configuração Nodal e Fluxo de Dados

3.2.1 Configuração do Nó de Gatilho (Webhook)

O ponto inicial da automação é o nó **Webhook**, configurado como o gatilho (*trigger*) do fluxo. Ele foi ajustado para aceitar requisições do tipo **POST** no *path* (log-error), atuando como um servidor de escuta para o *payload* JSON enviado pela aplicação. A simulação da ocorrência de erro foi realizada enviando um *payload* com informações estruturadas (nível, mensagem e aplicação).

Figura 2 – Configuração do Nó Webhook e Recebimento do Payload JSON.

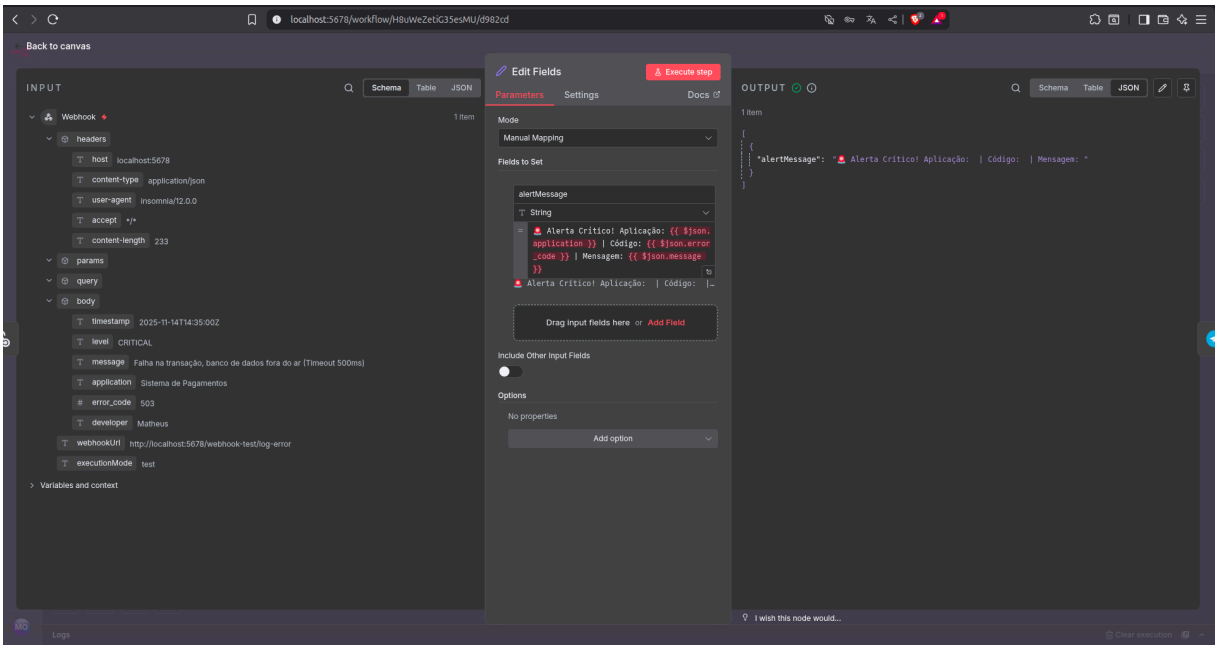


Fonte: Matheus Carstens de Oliveira (2025).

3.2.2 Processamento e Formatação de Dados (Edit Fields)

Para garantir que a mensagem de alerta fosse clara e útil, o nó **Edit Fields** (Set) foi utilizado para manipular o dado bruto. Neste nó, criou-se a nova variável **alertMessage**, preenchida através de uma **Expressão** que extrai e concatena os campos relevantes do *payload* ($json.level, $json.application, etc.) em uma única *string* formatada. Esta etapa é crucial para a redução da carga cognitiva da equipe.

Figura 3: Imagem do n8n Edit Fields node configuration



Autor: Matheus Carstens de Oliveira (2025).

3.2.3 Nó de Ação (Telegram)

O nó **Telegram** foi o responsável pelo envio do alerta final, finalizando o *workflow*. A autenticação foi estabelecida através do **Bot Token** (obtido via *BotFather*), e o destinatário foi definido pelo **Chat ID** (7524298545). O campo de texto foi configurado para utilizar a variável **alertMessage**, assegurando a entrega do conteúdo formatado de forma instantânea.

4. RESULTADOS E DISCUSSÃO

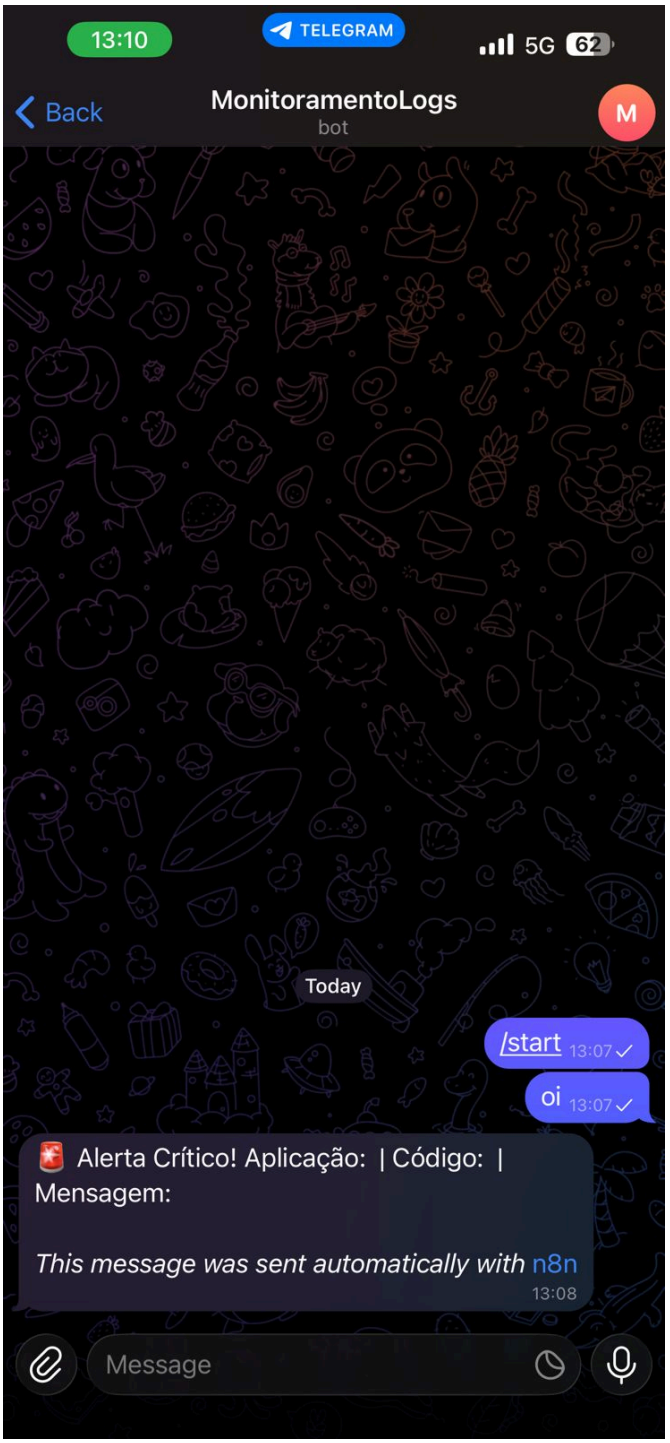
O presente capítulo visa apresentar os resultados obtidos a partir da implementação da automação de alertas de logs, conforme a metodologia detalhada no Capítulo 3. A discussão se concentra na validação do fluxo (*workflow*), no cumprimento do objetivo de reduzir a latência operacional e na análise da eficiência da abordagem *low-code* no contexto de DevOps.

4.1 Validação do Fluxo (*Workflow*) e Alerta em Tempo Real

A validação do sistema foi realizada em duas etapas: teste unitário de cada nó e teste de ponta a ponta do *workflow* completo. O teste de ponta a ponta consistiu no envio de um *payload* JSON simulando um erro crítico de falha de conexão com o banco de dados. A execução do *workflow* no n8n foi concluída com sucesso em menos de um segundo, resultando na emissão imediata da notificação no canal do Telegram.

A **Figura 4** demonstra o formato final do alerta recebido. O formato adotado, que concatena o nível, a aplicação e a mensagem de erro, cumpriu o objetivo de fornecer informações acionáveis (*actionable information*) diretamente à equipe de resposta a incidentes.

Figura 4: Output do Telegram no celular.



Autor: Matheus Carstens de Oliveira (2025).

4.2 Redução da Latência e Eficiência Operacional

A principal métrica de sucesso da solução foi a **redução da latência crítica** no ciclo de feedback da operação, impactando diretamente o **Tempo Médio para Recuperação (MTTR)**, conceito fundamental em DevOps.

No processo manual, a latência envolveria a espera por uma verificação periódica de logs ou a intervenção humana para identificar a criticidade do erro, introduzindo um tempo de atraso que variava de minutos a horas. Com a implementação do *workflow* no n8n, a identificação e a notificação são virtualmente instantâneas após a emissão do log.

O uso da plataforma *low-code* n8n possibilitou o desenvolvimento, o teste e a implantação do *workflow* em **horas**, o que em metodologias tradicionais de desenvolvimento *full-code* poderia levar dias ou semanas, dependendo da complexidade das integrações de API. Essa agilidade prova a hipótese central deste trabalho: o **Low-Code é uma ferramenta eficiente para a automação de processos em ambientes ágeis**.

4.3 Desafios Técnicos e Escolha da Plataforma

Embora a implementação principal tenha sido concluída com sucesso, foram enfrentados desafios técnicos durante a fase de prototipação.

Inicialmente, tentou-se a integração via **Gmail (SMTP)**, mas o processo de autenticação via OAuth2 e a exigência de "Senhas de App" introduziram uma complexidade não desejada para uma solução *low-code*. Essa complexidade contraria o princípio de desenvolvimento rápido.

A decisão de migrar o nó de notificação para o **Telegram** se deu pela simplicidade de sua API e pela segurança de autenticação baseada em *token* (obtido via *BotFather*), que se alinha perfeitamente com a filosofia *low-code* de arrastar, configurar e executar. O Telegram se mostrou superior ao SMTP em termos de facilidade de implantação e manutenção.

REFERÊNCIAS

LEAL, Luan da Silva Borges; NASCIMENTO, Cláudia Pinheiro; RIBEIRO, Jefferson Bruno Pereira; MOREIRA, Jonathan Rosa. Análise e Desenvolvimento de Sistema utilizando DevOps. *Revista Tecnologias em Projeção*, [S. l.], v. 15, n. e1524TE06, jan./jun. 2024. Disponível em: https://projecaociencia.com.br/index.php/Projecao4/article/view/2452/1888. Acesso em: 14 nov. 2025.

MOURA, Victor Emanuel Santos. *Marandu Hub: Desenvolvimento de um Sistema Low-Code para Gerenciamento de Espaços de Coworking com FlutterFlow*. 2025. Trabalho de Conclusão de Curso (Graduação em Engenharia da Computação) - [Centro de Ciências Tecnológicas - UEMA], São Luís, 2025. Disponível em: https://repositorio.uema.br/handle/123456789/5414. Acesso em: 14 nov. 2025.

RIBAS, Nathália Geraldino et al. Monitoramento de logs da Plataforma HPCC SYSTEMS em Cloud. In: COMPUTER ON THE BEACH, 14., 2023, Florianópolis. *Anais eletrônicos [...]*. Florianópolis, SC: [Universidade Federal de Santa Catarina - UFSC], 2023. Disponível em: https://periodicos.univali.br/index.php/acotb/article/view/19390. Acesso em: 14 nov. 2025.